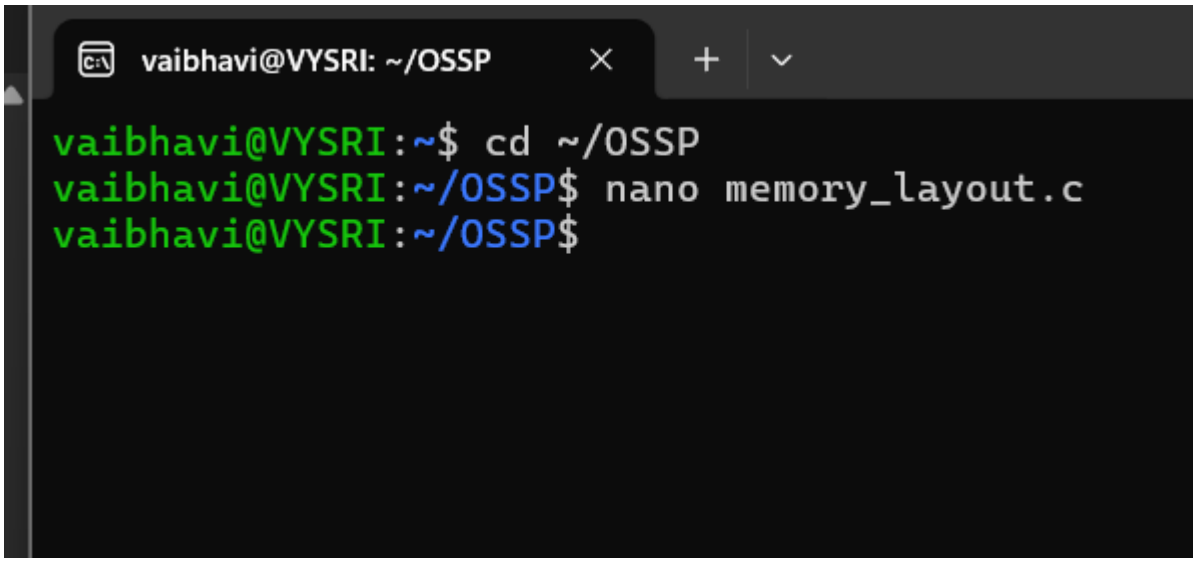
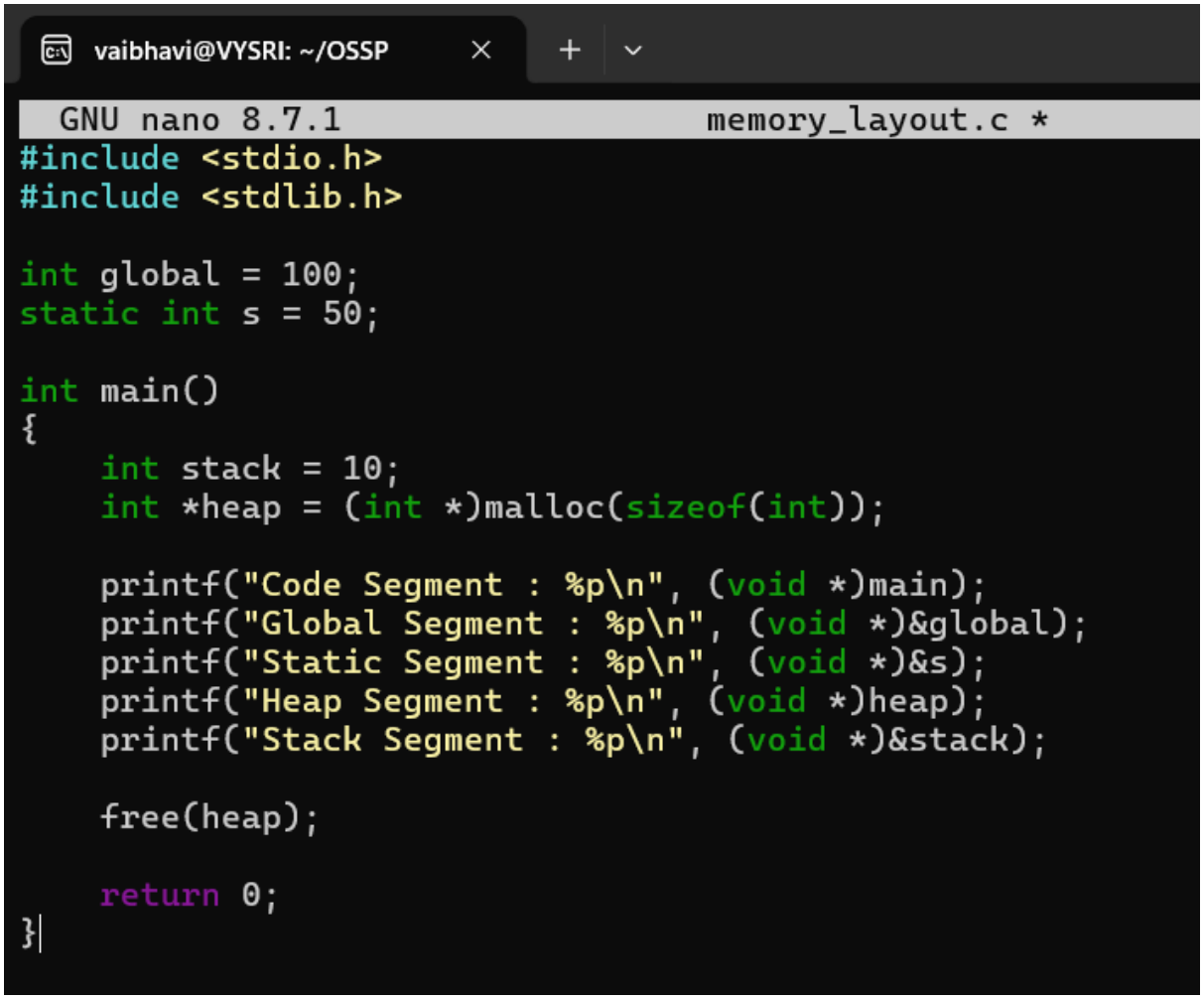
Name: Vaibhava Sree

Roll No: 2520030398

7. Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.





OUTPUT:

```
vaibhavi@VYSRI:~/OSSP$ gcc memory_layout.c -o memory_layout
vaibhavi@VYSRI:~/OSSP$ ./memory_layout
Code Segment : 0x631cfcfa91a9
Global Segment : 0x631cfcfac010
Static Segment : 0x631cfcfac014
Heap Segment : 0x631d21e37010
Stack Segment : 0x7ffc0a25ac1c
vaibhavi@VYSRI:~/OSSP$
```

8A. Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

```
Stack Segment : 0x7ffc0a25ac1c
vaibhavi@VYSRI:~/OSSP$ cd ~/OSSP
vaibhavi@VYSRI:~/OSSP$ nano allocation.c
vaibhavi@VYSRI:~/OSSP$
```

vaibhavi@VYSRI: ~/OSSP

```
 GNU nano 8.7.1                    allocation.c *
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *a, *b;

    a = (int *)malloc(5 * sizeof(int));

    b = (int *)calloc(5, sizeof(int));

    a = (int *)realloc(a, 10 * sizeof(int));

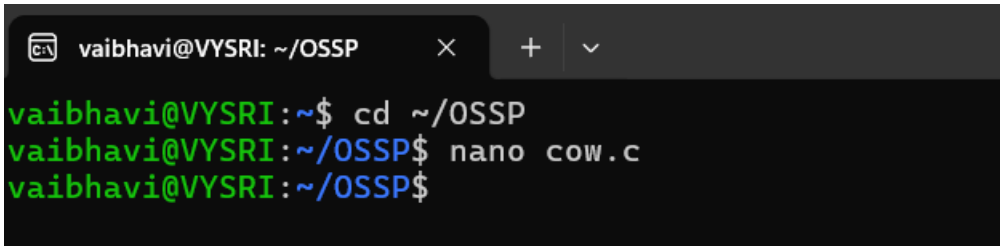
    free(a);
    free(b);

    return 0;
}
```

OUTPUT:

```
vaibhavi@VYSRI:~/OSSP$ valgrind --version
valgrind-3.26.0
vaibhavi@VYSRI:~/OSSP$ cd ~/OSSP
vaibhavi@VYSRI:~/OSSP$ gcc allocation.c -o allocation
vaibhavi@VYSRI:~/OSSP$ ./allocation
vaibhavi@VYSRI:~/OSSP$ valgrind --leak-check=full ./allocation
==1231== Memcheck, a memory error detector
==1231== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==1231== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==1231== Command: ./allocation
==1231==
==1231==
==1231== HEAP SUMMARY:
==1231==     in use at exit: 0 bytes in 0 blocks
==1231==   total heap usage: 3 allocs, 3 frees, 80 bytes allocated
==1231==
==1231== All heap blocks were freed -- no leaks are possible
==1231==
==1231== For lists of detected and suppressed errors, rerun with: -s
==1231== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
vaibhavi@VYSRI:~/OSSP$
```

8b. Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.



vaibhavi@VYSRI: ~/OSSP

GNU nano 8.7.1 cow.c

```c
#include <stdio.h>
#include <unistd.h>

int main()
{
    int x = 100;

    if(fork() == 0)
    {
        printf("Child Before=%d\n", x);

        x = 500;

        printf("Child After=%d\n", x);
    }
    else
    {
        sleep(2);

        printf("Parent=%d\n", x);
    }

    return 0;
}
```

OUTPUT:

```
vaibhavi@VYSRI:~/OSSP$ nano cow.c
vaibhavi@VYSRI:~/OSSP$ gcc cow.c -o cow
vaibhavi@VYSRI:~/OSSP$ ./cow
Child Before=100
Child After=500
Parent=100
vaibhavi@VYSRI:~/OSSP$
```